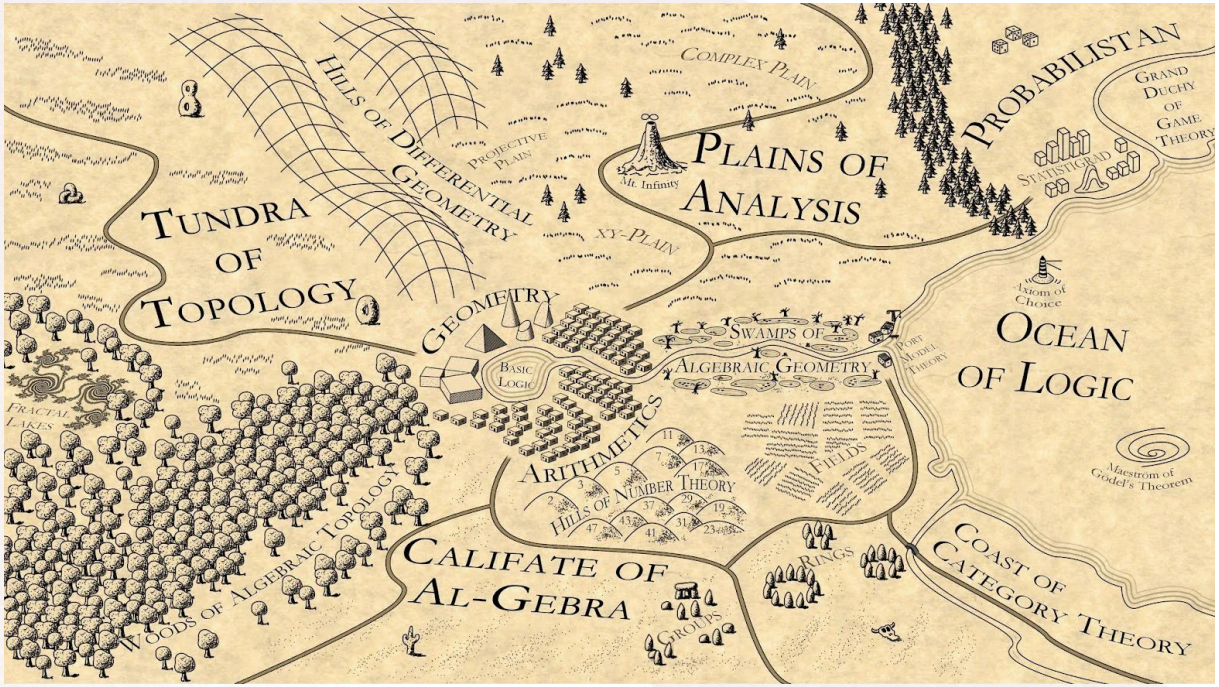
SymPy: symbolic calculation

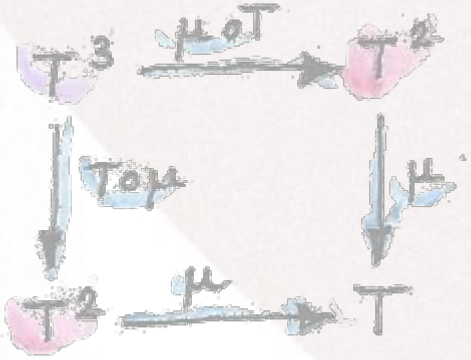


P. BONNARD & R. CARBOU

Content and motivation

- Introduction
  - Symbolic calculation
  - SymPy library
- Core instructions
  - Algebraic operators
  - Differential calculation
  - Integral calculation
- Advanced operators
  - Differential equations
  - Laplace transformation
  - Fourier transformation
  - Matrices and vectors
  - AST and expression

INTRODUCTION



Introduction: symbolic calculation

- Symbolic computation handles **non-numerical values**, this means **symbols** like in algebra.
- It emphasizes on term **rewriting** over digital **evaluation**.
  - Symbols or expressions as rewritable terms
  - Values imply a loss
  - Symbols are more abstract, values more concrete
- Getting formulas/expressions as the results of computations is usually desirable when possible.

Symbolic calculation examples

- What is the value of 3/9?
  - Symbolic systems answer 1/3
  - Value systems answer 0.333333…
  - Loss of information: 0.333333… * 3 is not necessarily equal to 1 depending on the precision of number representations

- What is the value of sqrt(2)^2?
  - A symbolic system knows that **square** is the inverse of **sqrt**. After simplification, it replies 1.
  - A digital system evaluates sqrt(2) as a number and then applies **square**. It may be close to, but different from 1, depending on precision.

Symbolic calculation and variables

- Symbolic calculation keeps the variable as a symbol:
  - (x+y) + (x-y) is simplified as 2x
- Numerical calculation
  - Given a valuation, e.g. x=3, y=7
  - The evaluation returns 6

What is SymPy

- Python library for symbolic calculation
- Free: licensed under BSD
- Reference: https://www.sympy.org
- Documentation:

  https://docs.sympy.org/latest/index.html
- Started in 2005
- Features ranging from basic symbolic arithmetics to calculus, algebra, discrete mathematics, and quantum physics

SymPy features I

- Core: arithmetic, function, pattern matching…
- Polynomials
- Calculus: limit, differentiation, integration, Taylor
- Solver: linear equation, algebraic equation, differential equation
- Discrete maths: summation, product, logic…
- Matrices: calculus, eigenvalue, determinant
- Geometry: figures, tangency...

SymPy features II

- Plotting: 2D, 3D, coordinate models
- Physics: Classical and Quantum mechanics, linear control…
- Statistics: distributions, probabilities
- Combinatorial: permutation groups...

Projects using SymPy

- Cadabra: Tensor calculation
- ChemPy: Chemistry
- EinsteinPy: General relativity
- PyDy: Multi-body dynamics
- Quantum Programming
- Spyder: Scientific framework for calculation
- Symbolic statistical modelling
- ...

CORE INSTRUCTIONS



Importation

- The instructions come from the package SymPy
- To clearly separate Python expressions from SymPy ones, we prefix them with the **sym** alias (alternatively often used: **sp**).
  - import sympy as sym
- Cumbersome and confusing but still possible and more compact:
  - from sympy import *

Number

- Python represents 2/5 as a float
- SymPy can represent it as a rational:
  - sym.Rational(2, 5)
- Likewise, for an integer:
  - sym.Integer(2)
- Function **sympify**() (**S**() shortcut) for parsing:
  - sym.sympify("3.1415") or sym.S("3.1415")
- Mathematical constants:
  - sym.pi, sym.E or sym.I (why those capitals?...)

Operation on number

- Standard binary maths operators, overridden:
  - Addition and subtraction: + -
  - Multiplication and division: * /
  - Exponentiation: **
- Unary functions: sqrt, exp, sin, cos, tan...
- Examples:
  - (sym.pi + 1/2)**3
  - sym.sqrt(2)
  - sym.sin(sym.pi/3)**2 + sym.cos(sym.pi/3)**2

Number evaluation

- Function **evalf**(): obtaining a float approximation of a number
  - sym.pi.evalf()
- Specify digits of the approximation:
  - sym.pi.evalf(100)
- “Simplify” an expression:
  - sym.simplify(*expression*)
  - *expression*.simplify()
  - sym.simplify(sym.sin(math.pi/10)**2)
- No confusion! **sympify** ≠ **simplify**!!!
- The result ...may be frustrating.

Exercise 1.1: Number



Symbol declaration

- Symbols used In SymPy **must** be declared:
  - alice, bob = sym.symbols("x y")
  - **Preferably**: x,y = sym.symbols("x y")
- Types might be indicated
  - i,j,k = sym.symbols("i j k", integer=True)
- Functions should be indicated that way:
  - f,g = sym.symbols("f g", cls=Function)
- Implicit declaration when an expression is parsed with the **sympify** function:
  - *expr* = sym.sympify("x+y")

Expression

- An expression includes a valid combination of constants, variables and symbols, unary and binary operators:
  - expr = 2 * x + sym.cos(x) * 2 + y**3 - x
- The **simplify**() function tries to perform simplification techniques on an expression:
  - sym.simplify(expr) → x + sym.cos(x)*2 + y**3

Function definition

- Defining a SymPy function is achieved through a Python function declaration

```
def f(arg):
    return arg**2 + 1
```

- Rather **do not** specify types in the signature
- Applying function f to a symbol as f(x) is evaluated by SymPy as
  - x**2 + 1
- Applying function f to a constant, e.g. f(2), is evaluated by Python as a standard value
  - f(2) → 5

Signature and variables

- Generally speaking, a signature Σ (or "ranked alphabet") is composed of:
  - A set of symbols **F** (a constant c, a function f…)
  - A function **Ar**: F → N, giving a unique *arity* to each symbol
    - Ar(c) = 0 (a constant ...takes no argument!)
    - Ar(f) = 1 (for a unary function)
- Variables in the set V are...
  - ...defined separately from Σ
  - ...terms for expression processing/rewriting

Terms of the signature

- The set of terms $\theta(\Sigma,V)$ over Σ and the variables V is **recursively** defined as:
  - $\forall\, x \in V \Rightarrow\ x \in \theta(\Sigma,V)$
  - $\forall\, c \in F,\ Ar(c)=0 \Rightarrow c \in \theta(\Sigma,V)$
  - $\forall g \in F$ such that $Ar(g)=n$,

    $\forall t_1,\ldots,t_n \in \theta(\Sigma,V)^n \Rightarrow g(t_1,\ldots,t_n) \in \theta(\Sigma,V)$

Terms example

- Given $\Sigma_1 = (F_1, Ar_1)$ and $V_1 = \{x, y\}$
  - $F_1 = \{0, 1, pi, +, -\}$
  - $Ar_1(0) = Ar_1(1) = Ar_1(pi) = 0$
  - $Ar_1(+) = Ar_1(-) = 2$
- A **few** terms of $\theta(\Sigma_1, V_1)$...
  - x, y
  - 0, 1, pi
  - 0+pi, x+1, x-y, (pi-y) + ((x+1)-pi)…
- Now, propose a minimal Σ to build the set N.

Ground terms

- A **ground term** is a term that does not contain any variable.
- The set of ground terms $\theta(\Sigma)$ is equal to $\theta(\Sigma, \emptyset)$
- Trivial property: $\theta(\Sigma) \subset \theta(\Sigma, V)$
- Examples:
  - $\{1, pi\} \subset \theta(\Sigma_1)$
  - $\{1 + pi, pi - 1 + 0\} \subset \theta(\Sigma_1)$
  - $\{1 + pi - x, x + y\} \not\subset \theta(\Sigma_1)$

Substitution with equality

- Substitution property: If two things are equal, then any property of one is a property of the other.
- It is sometimes referred to as "**Leibniz's law**".
- Formally: $\forall$ a, b, $\forall$ ϕ (x), a=b $\Rightarrow$ (ϕ(a) $\Rightarrow$ ϕ(b))
- Application:
  - X = 3 Y,
  - ϕ (X) = (2X = 6Z) $\Rightarrow$ ϕ (3Y) = (6Y = 6Z)

Substitution over variables

- A substitution σ is a function from **variables** to **terms**:
  - $\sigma : V \rightarrow \theta(\Sigma, V)$
- The domain **dom** of σ is the **subset** of variables **actually** replaced, usually finite.
  - $dom(\sigma) = \{v \in V \mid \sigma(v) \neq v\}$
- The substitution is notated as:
  - $dom(\sigma) = \{v_1, ... v_p\}$
  - $\sigma = [v_1 \rightarrow t_1, \ldots, v_p \rightarrow t_p]$
- Example:
  - $\sigma_1 = [X \rightarrow 1, Y \rightarrow 1 + X]$

Substitution applied to a term

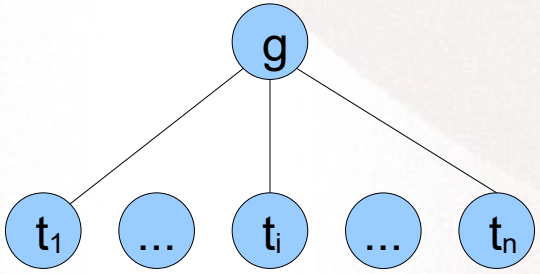
- Given a substitution $\sigma: V \rightarrow \theta(\Sigma,V)$...
- ...σ could be **extended** to $\sigma_e: \theta(\Sigma,V) \rightarrow \theta(\Sigma,V)$ in order to be applied to any **term** t:
  - $\forall t \in \theta(\Sigma), \sigma_e(t) = t$ (ground terms...)
  - $\forall g \in F$ such that $Ar(g)=n$, $\forall t_1,\ldots,t_n \in \theta(\Sigma,V)^n$
  - $\sigma_e(g(t_1,\ldots,t_n)) = g(\sigma_e(t_1),\ldots,\sigma_e(t_n))$
- Applying a substitution σ_e to any term is written:
  - $dom(\sigma) = \{v_1,\ldots,v_p\} \subset V$, $t \in \theta(\Sigma,V)$
  - $\sigma_e(t) = t[v_1 \rightarrow t_1, \ldots, v_p \rightarrow t_p]$

Term substitution examples

- $x\,[x \rightarrow 4\,] = 4$ (**evaluation**!)
- $(x + y)\,[x \rightarrow 4, y \rightarrow 2] = 4 + 2$
- $(x-z)\,[z \rightarrow x+2] = x - (x+2)$
- $(1+(x-y))\,[x \rightarrow 0, y \rightarrow t-1] = 1+(0 - (t-1))$

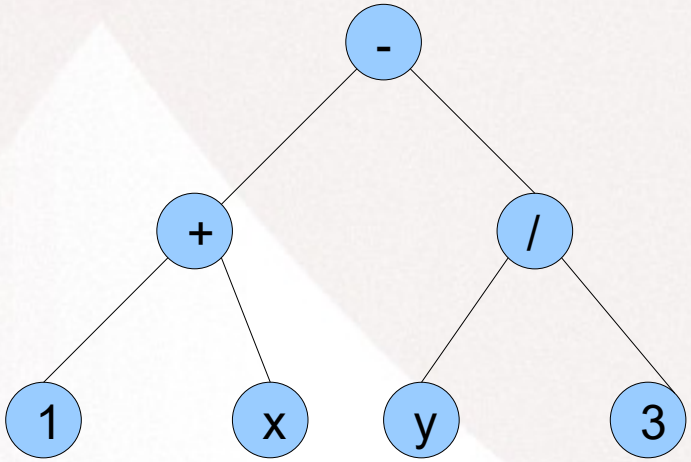
Term represented as tree

- A term $t \in \theta(\Sigma,V)$ could be represented as a **tree** where constants and variables are leaves, and other symbols are intermediate nodes.
- At intermediate level,
  - $g \in F$, $Ar(g) = n$
  - $t = g(t_1,\ldots,t_n)$, $\forall t_1,\ldots,t_n \in \theta(\Sigma,V)^n$
  - t is represented by the following subtree:



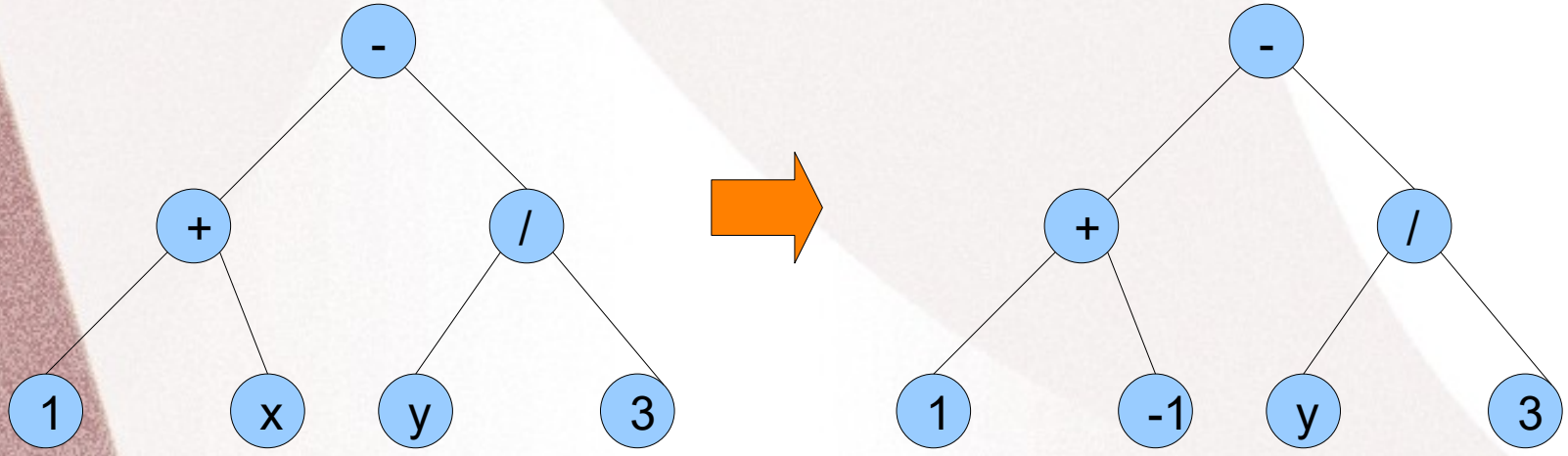
Term as tree example

- The expression: (1+x) - (y/3) is represented by the following tree:



Substitution over tree expression

- Applying the simple substitution $[x \rightarrow t_x]$ to a tree t is equivalent to replacing every leaf by the term t_x
- Example with t = (1+x) - (y/3) and $[x \rightarrow -1]$
  - $t[x \rightarrow -1] = (1-1) - (y/3)$

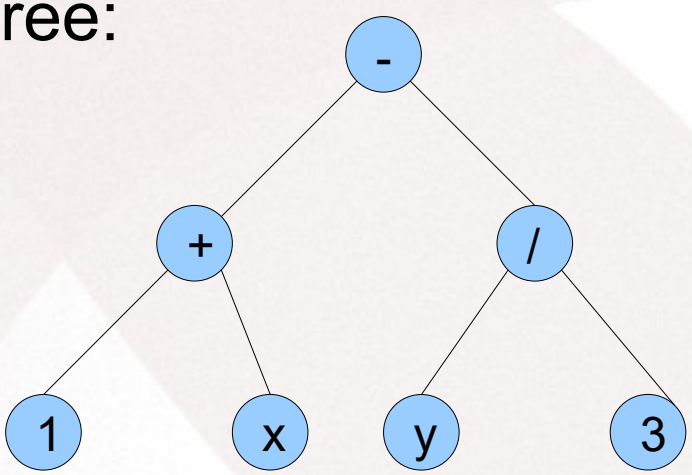


Observation about substitution and tree

- By default, substitutions do not apply any simplifications (e.g.: 1-1, x+0), although the latter might be specified in substitutions.
- Substitutions are often extended to meta-variables, as a full map $\theta(\Sigma,V) \rightarrow \theta(\Sigma,V)$:
  - $[2+x \rightarrow y,\ x+y \rightarrow 0]$
  - $[0*_ \rightarrow 0,\ _*0 \rightarrow 0\ ]$
- **Polish notation** helps to represent trees as an equivalent list of symbols:
  - The structure of the tree is better shown
  - Example: (+ 1 x)

Polish notation example

- The expression (1+x) - (y/3) is represented as a tree:



- As a polish notation expression with or without parenthesis:

→ (- (+ 1 x) (/ y 3))

→ - + 1 x / y 3

Substitution in SymPy I

- When functions are invoked, SymPy implicitly substitutes the formal parameters with input arguments (see earlier):
  - f(x) → x**2 +1
- Function **subs**() explicitly substitutes sub-expressions of an expression:
  - Applied on an expression
  - A map defines how variables are rewritten
  - z = sym.cos(x)
  - z.subs({x:sym.pi}) → cos(pi) → -1

Substitution in SymPy II

- Substitution is a **key operation** of symbolic calculus and rewriting processing
- Function call could be evaluated as a substitution (list, functional language)
- Might be applied on several variables at the same time:
  - z = x + 2*y
  - z.subs({ x:3, y:t+2}) → 3 + 2*(t+2)
- Might be applied on complex terms other than variables (pattern matching on the left part)
  - z.subs({ x:3, 2*y:t}) → 3 + t

Exercise 1.2: Expression



Polynomials

- SymPy helps to define polynomials or multinomials, given a set of symbols or variables or constants:
  - p1 = a * x**2 + 3 * x + c
  - p2 = a * x**3 * y**2 + 10 * x * y
  - p3 = (a * x + 1) * (b*x - 2)

expand/factor polynomials

- **expand**() function expresses the polynomial in its sum form:
  - sym.expand((x+1)*(x-1)) → x**2-1
- **factor**() function tries to express the polynomial as a product:
  - sym.factor(x**2-1) = (x+1)*(x-1)
  - It may be useful to find roots of the polynomial!
- **simplify**() function tries to factor the polynomial:
  - sym.simplify(x**2-1) = (x+1)*(x-1)

Collecting polynomials

- **collect**() function helps to group an expression according to ordered powers of a variable:
  - e = x*y + x - 3 + 2*x**2 + x**3 - z*x**2
  - sym.collect(e, x) → x**3 + (2-z)*x**2 + (y+1)*x - 3
- **coeff**() function returns the coefficient of a particular term, given its power:
  - sym.coeff(3*x**2 + 12, 2) = 3

Rational function

- Expression as fraction of symbols or polynomials
  - Example: (x + 3) / (y**2 - 1)
- **together**() function calculates the addition of 2 fractions put on a common denominator:
  - t = a / b + c / d
  - sym.together(t) = (a*d + b*c) / (b*d)
- **apart**() function helps to divide and simplify 2 rational expressions as a **sum** of fractions:
  - h = (x**2 + x + 4) / (x + 2)
  - sym.apart(h) = x - 1 + 6/(x + 2)

Exercise 1.3: Algebraic Expression



Logarithm and exponential

- By default, SymPy assumes complex inputs for **log**() and **exp**()
- Euler constant sym.E is equivalent to the standard exponential series at 1
- sym.exp(x) is equivalent to sym.E**x
- Possible to **constrain** symbols to integer or real or positive when declared:
  - x,y = sym.symbols("x y", positive=True)

Complex expression

- sym.I is the imaginary number:
  - sym.I * sym.I = -1
- Complex numbers are defined this way:
  - z = 4 + 3 * sym.I
- Standard complex functions:
  - re, im,
  - arg, abs (norm)
  - conjugate
- Euler's exponential form:
  - sym.exp(I*x)

Euler's exponential form

- Classic relation between complex exponential and trigonometric functions ("sym" prefix purposely **omitted** from now, for the sake of clarity)
  - exp(I*x) = cos(x) + I*sin(x)
- Expansion possible when indicated:
  - expand(exp(I*x), complex=True)
  - cos(x) + I*sin(x)

Exercise 1.4: Complex expression



Simplification

- There is no unique definition of what the “simplest” form of an expression is…
- **simplify**() tries several methods, sometimes failing to produce what **you** have in mind.
- **simplify**() on polynomials tries to factorize them when possible
- There are specialized simplifications such as **trigsimp**() ...for trigonometry.

Equality

- **Syntactical** equality operator **==** (Python) checks that 2 expressions are **the same tree** (i.e. AST).
  - x + y == x + y is **true**
  - x + y == y + x is **false**
  - x * (1+y) == x + x*y is **false**
- **simplify**() function may transform a form in the other one, although it is not always true:
  - simplify(x*(1+y)) == x + x*y **might** be true
- **Not reliable** to establish **semantical** equality!

Semantic equality

- To check that 2 expressions P, Q are **semantically** the same, one should check that their difference P - Q = 0.
- **simplify**() must be applied on the difference.
- Then, check it is syntactically equal to 0:
  - simplify(P - Q) == 0
- Example:
  - sin(x)**2 + cos(x)**2 == 1 is **false**
  - ...but simplify(sin(x)**2 + cos(x)**2 - 1) = 0

Eq form for equation

- An equation is represented with the **Eq**() instruction.
  - Keep the same form and semantics as the mathematical definition (avoiding sign error when converting into a difference!)
- Example:
  - x + y = 10 represented as Eq(x+y,10)

Algebraic equation solver

- **solve**() function might be able to find symbolic roots of polynomial equations
  - No general method for polynomials of order ≥ 5 (Abel-Ruffini, 1824)
- Solving x**2 = y:
  - sym.solve(x**2 - y, x)
  - Returns -sqrt(y) and sqrt(y)
- Keeping the equality form by using Eq:
  - expr = sym.Eq(x**2, y)
  - sym.solve(expr, x)

Restricting the domain

- By default, solutions are complex numbers:
  - sym.solve(sym.Eq(x**4, 16), x)
  - Returns { -2, 2, -2*I, 2*I }
- If needed, limit the domain to, e.g., real numbers. At the time of symbol declaration:
  - x = sym.symbols("x", real = True)
  - sym.solve(sym.Eq(x**4, 16), x)
  - Returns { -2, 2 }
- **solveset**() restricts the domain explicitly. Expressions are implicitly equated to 0:
  - solveset(Eq(x**4, 16), x, domain=S.Reals)
  - solveset(x**4 - 16, x, domain=S.Reals)

Exercise 1.5: Solving polynomial equation



Extended solving

- **solveset**() is not limited to polynomials. It may be applied to trigonometric or other functions.
- The solving might be limited to an arbitrary domain set including intervals:
  - solveset(sin(x), x, Interval(-pi, pi))
  - Returns { -pi, 0, pi }
- The solution set might be infinite, represented as a **comprehension set**:
  - solveset(sin(x), x)
  - {2*n*pi | n in Integer} U {2*n*pi+pi | n in Integer}
- Most of equations **are not** symbolically solvable.

Exercise 1.6: Solving equation



Limit

- sym.oo is the infinity symbol
  - oo + 1 = oo
  - n < oo
  - 1 / oo = 0
- The infinity symbol is useful for defining limits, integrations, series…
- **limit**() function calculates the limits of sequences or functions when reaching intermediate or infinitely small or large values.

Examples of limit

- limit(1/x, x, oo) $\lim_{x \to \infty} \frac{1}{x}$
- limit(1/x, x, 0, dir="+") $\lim_{x \to 0^+} \frac{1}{x}$
- limit(1/x, x, 0, dir="-") $\lim_{x \to 0^-} \frac{1}{x}$

Derivative

- SymPy is able to calculate derivatives...
  - ...as a limit of rate of change
  - ...as a symbolic calculation

$$\frac{df}{dx} = \lim_{dx \to 0} \frac{f(x+dx) - f(x)}{dx}$$

- **diff**() computes the derivative of an expression:
  - diff(sin(x), x) = cos(x)
- SymPy knows the basic laws:
  - (f+g)' = f' + g'
  - (fg)' = f'g + fg'
  - (f∘g)' = (f'∘g)*g'

Extended derivation

- **diff** function calculates the Nth derivative:
  - diff(sin(x), x, 2) = -sin(x)
- Partial derivations are possible by indicating the derivation variable:
  - diff(y*sin(x), x) = y*cos(x)
  - diff(y*sin(x), y) = sin(x)
- Gradient calculation:
  - Compute the gradient vector “manually”
  - Use **sympy.tensor.array.derive_by_array**

Exercise 1.7-1.8: Limit and derivative



Integration

- SymPy calculates integrals with function **integrate**().
- Indefinite integrals or primitives:
  - integrate(x**3, x) for primitives $\int x^3 dx$
  - No integration constant added
- Definite integrals when bounds are specified
  - integrate(x**3, (x, a, b)) $\int_a^b x^3 dx$
- Bounds might be infinite:
  - integrate(1/t**2, (t, 1, oo)) $\int_1^\infty \frac{1}{t^2} dt$

Exercise 2.1: Integration



Sequence

- Sequences are applications defined on natural numbers. One can see them as a list of indexed values a_n.
- SymPy helps to calculate their limits and properties when they are explicitly defined:
  - a_n = 1 / factorial(n)
- Sequences can be defined by recurrent equations between their successive terms:
  - u(n+2) = u(n+1) + u(n)
- Python functions help to define sequences.

Fibonacci sequence

```
def fib_n_2(fib_n_1, fib_n):
    return fib_n_1 + fib_n


fibs = [1,1]
n = 10
for i in range(n):
    fibs.append(fibs[-1], fibs[-2])
```

- The list of terms helps to identify the trends (periodicity, bounds…) of the sequence

Python program and SymPy

- Python program (functions, classes) leverages SymPy features:
  - Symbolic expression creation (recursive function)
  - Composition of SymPy processing
  - Iterating/Analyzing SymPy results

Exercises 2.2-2.3: Sequence



Series: summation

- Series are sums of sequences
- Function **summation**() similar to integration but applied to discrete sequence a_n

$$\sum_{n=0}^{\infty} a_n$$

- **summation**(a_n, (n, 0, sym.oo))
- Computing formal and infinite sums
  - Be careful with convergence
  - Example equivalent to e:

$$\sum_{n=0}^{\infty} \frac{1}{n!}$$

- Function **evalf**() helps to approximate the series as a float

Power series

- Power series useful to approximate functions where terms are powers of the variable x.
- Also named “Taylor series” developed at 0 or at any real c of the definition domain.

$$f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(0)x^n}{n!}$$

$$g(x) = \sum_{n=0}^{\infty} \frac{g^{(n)}(c)(x-c)^n}{n!}$$

$$e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!}$$

Power series II

- Taylor series are useful to compute values of “regular enough” functions by using limited polynomials
- Difference between **symbolic** and **numerical** series
- Infinite symbolic summation
  - summation(x**n / **sym**.factorial(n), (n, 0, oo))
  - formally equal to e^x
  - x is a SymPy symbolic variable
- Limited numeric summation for approximation
  - sum([y**i / **math**.factorial(i) for i in range(0,20)])
  - y is a standard Python variable

Power series III

- **series**() helps to get power Taylor series of an expression at value c, up to a maximum power.
  - series(sin(x), x, 0, 10)
  - series(cos(x)/x**2, x, 1, 15)
  - series(sqrt(a + b*x**3), 0, 10)
- A remainder $o(x^p)$ is added to the terms.
  - For numeric evaluation, it should be removed with the **removeO**() function.
- Returns a limited power series that may be approximated at a given value by substitution:
  - series(sin(x), x, 0, 10).subs({x:3})

Exercises 2.4-2.5 Series



Series and code

- Series calculation might be linked to program
  - Taylor approximation
  - Derivation / integration
  - Error calculation…
- Taylor series for pi approximation

Exercises 2.6-2.7: Series



Usage of series

- Series are widely used in mathematics
- Approximation of constants such as e or pi
- Approximation of functions (sin, cos)
- Differential equations solving
  - “PDE” (Partial Differential Equations)
  - “ODE” (Ordinary Differential Equations)

Exercise 2.8: Calculating pi



Polynomial and code

- Code helps to define complex polynomial sequences with algebraic/analytical properties
- It permits to introduce primitive and derivative calculation as well as problem solving
- The following example shows how to calculate Bernoulli polynomials as a recurrent sequence

Exercise 3.1: Bernoulli polynomials



ADVANCED OPERATORS



Differential equations

- Differential equations are functional equations where a (partial) derivative of any order occurs
- Main concerns for physics and engineering
- SymPy helps to solve **some** of them
- The **frequencies** space might help to find solutions (Fourier/Laplace transformations)

Simple oscillator

- Example: a spring acting on a mass
- We easily obtain the following ODE:

$$-kx(t) = m\frac{d^2}{dt^2}x(t)$$

- The resolution in SymPy just requires to...
  - declare functions and variables with the **symbols** function (or **Symbol**/**Function** constructors)
  - declare the ODE by constructing left and right parts with the **diff**() function
  - activate the resolution with the **dsolve**() function

Simple oscillator II

- More precise solutions found when variables and constants are correctly typed:
  - m = sym.Symbol("m", positive=True)
  - f = sym.Function ("f", real=True)
- **Reminder**: Symbols are considered as complex numbers by default.
- Function f occurs in the ODE as f(t) or **diff**(f(t)).

Function dsolve

- Function **dsolve**() returns an equality as solution
  - sol = dsolve(Eq(diff(x(t), t), …), x(t))
  - sol.rhs to obtain the right part
- Accept initial/boundary conditions with the **ics** parameter:
  - dsolve(…, x(t), ics={x(0):L, x(t).diff(t).subs(t, 0):0})
- Other solving methods are available

Exercise 3.2: Simple oscillator



ODE and Laplace transform

- Laplace transform: useful to solve ODE
  - Optimal control
  - Electrical engineering
- Linearity
- Derivation/Integration translation

$$L\{f(t)\} = F(s) = \int_0^\infty e^{-st} f(t) dt$$

ODE and Laplace transform example

$$y''(t) + ay'(t) + by(t) = g(t)$$
$$y(0) = c$$
$$y'(0) = d$$
$$L\{y''(t)\} + aL\{y'(t)\} + bL\{y(t)\} = L\{g(t)\}$$
$$s^2Y(s) - sy(0) - y'(0) + a(sY(s) - y(0)) + bY(s) = G(s)$$
$$s^2Y(s) - cs - d + asY(s) - ac + bY(s) = G(s)$$
$$Y(s)(s^2 + as + b) = G(s) + (cs + d + ac)$$
$$Y(s) = \frac{G(s) + cs + d + ac}{s^2 + as + b}G(s))$$
$$y(t) = L^{-1}\{Y(s)\}$$

Laplace transform in SymPy

- **laplace_transform**() function
  - sym.laplace_transform(f(t), t, s)
- **inverse_laplace_transform**() function
  - sym.inverse_laplace_transform(Y(s), s, t)
- Solving an ODE calls the following steps:
  - Apply **laplace_transform()** to the equation to obtain Y(s) equation in frequency domain
  - Solve Y(s) in frequency domain with **solve()**
  - Calculate y(t) with **inverse_laplace_transform**()

Exercises 3.3 – 3.4: Laplace transform



Fourier transform

- Fourier transform: often used in the industry
  - Signal processing
  - ODE / PDE solving
  - Media compression (jpeg, mpeg)
- SymPy helps to solve PDE by providing Fourier transform calculation
- Any periodic and regular enough function might be seen as a **trigonometric series**:

$$c_n(f) = \frac{1}{T}\int_0^T e^{-i\frac{2n\pi}{T}t} f(t)dt$$

$$f(t) = \sum_{n\in\mathbb{Z}} c_n(f) e^{i\frac{2n\pi}{T}t}$$

Fourier series

- The function **fourier_series**() computes the Fourier series expansion, providing its limits
  - s = sym.fourier_series(x**2, (x, -pi, pi))
- The function **truncate**() ...truncates the Fourier series up to a given order
  - s.truncate(n=5)

$$a_n(f) = \frac{2}{T}\int_0^T cos(\frac{2n\pi}{T}t)f(t)dt$$

$$b_n(f) = \frac{2}{T}\int_0^T sin(\frac{2n\pi}{T}t)f(t)dt$$

$$f(t) = \frac{a_0}{2} + \sum_{n=1}^{\infty}(a_n(f)cos\frac{2n\pi}{T}t + b_n(f)sin\frac{2n\pi}{T}t$$

Fourier transform in SymPy

- The function **fourier_transform**() computes the complex frequency transform of a function
  - sym.fourier_transform(t**2, t, k)
- The function **inverse_fourier_transform**() calculates the temporal function of a frequency function F(k)
  - sym.inverse_fourier_transform(exp(-(pi*k)**2), k, t)

$$F(k) = \int_{-\infty}^{\infty} e^{-i2k\pi t} f(t) dt$$

$$f(t) = \int_{-\infty}^{\infty} e^{i2k\pi t} F(k) dk$$

SymPy graphical representation

- The function **plot**() represents as graphics a list of functions, given the selected variable and its minimum and maximum values.
  - plot(t, t**2, (t, 0, 10))
- It depends on the **matplotlib** Python library.
- **plot3d**(), **plot3d_parametric_line**(), **plot3d_parameteric_surface**() represent 1D, 2D varieties in a 3D graphics.

Exercises 3.5 – 3.6: Fourier transform

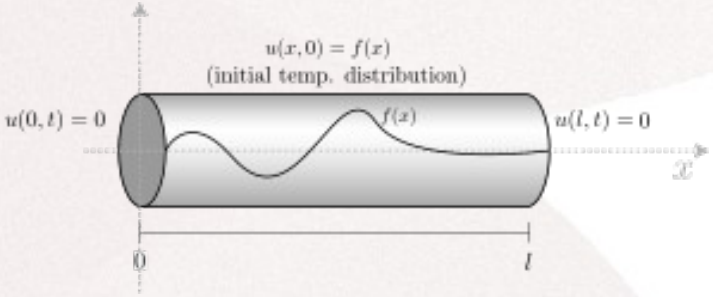


Fourier transform and PDE

- The **heat problem** tries to find temperature's temporal evolution of a piece of metal. Solved by **Fourier** in 1822 who introduced trigonometric series
- Physical models lead to a PDE where figures a **laplacian** over spatial variables and a thermal diffusion constant.

$$\frac{\partial u(t,x,y,z)}{\partial t} = a\left(\frac{\partial^2 u(t,x,y,z)}{\partial x^2} + \frac{\partial^2 u(t,x,y,z)}{\partial y^2} + \frac{\partial^2 u(t,x,y,z)}{\partial z^2}\right)$$

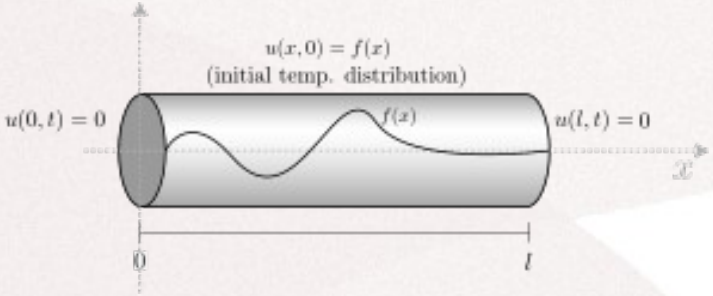
1D Heat problem



- We limit the problem to a 1D metal bar of length L with the following initial and boundaries conditions.
- The system of equations is:

$$\frac{\partial u(t,x)}{\partial t} = a\frac{\partial^2 u(t,x)}{\partial x^2}$$
$$u(0,x) = f(x)$$
$$u(t,0) = 0$$
$$u(t,L) = 0$$

1D Heat solution



- To solve this problem, we must:
  - Separate x and t variables
  - Use a Fourier spatial series for f(x)
- The solution of that problem is given by:

$$u(t,x) = \sum_{n=1}^{\infty} F_n sin\left(\frac{n\pi x}{L}\right) e^{-\frac{n^2\pi^2}{L^2}at}$$

$$F_n = \frac{2}{L}\int_0^L f(x) sin\left(\frac{n\pi x}{L}\right) dx$$

Exercises 3.7: Heat problem



Discretizing the PDE

- ODE or PDE often **too complex** to be formally solved
- SymPy provides the set of functions to approximate the derivatives by finite differences
  - Discretizing time and space variables
  - Algebraic resolution or approximation
- In the sympy.calculus package
  - **apply_finite_diff**(),
  - **differentiate_finite**()
  - ...

ODE/PDE numerical solving

- Set up the ODE/PDE in SymPy
- Find formal properties and approximation (series, Fourier…) with symbolic calculus
- Symbolic level **more informative** than numerical
- Convert it to a numerical function
  - Using **lambdify**() function
  - f = sym.lambdify((x, t), x**2/t)
- Solve the initial problem by calling an external numerical solver (for example, SciPy **solve_ivp**)

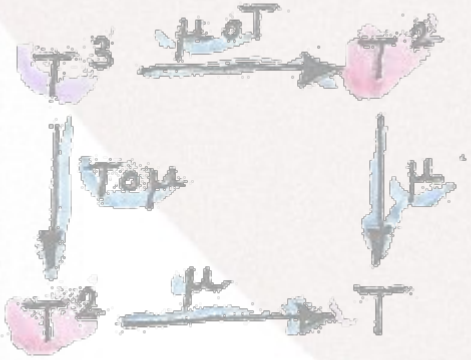
Other functions of calculus

- Monotonic:
  - **is_decreasing**(), **is_monotonic**()…
- Singularities: poles…
- Domain, codomain
  - **continuous_domain**()
  - **function_range**()
- Periodicity, minimum, maximum
- ...

Exercises 3.8: Chemical kinetics



LINEAR ALGEBRA



Vectors

- Vectors are available in SymPy
- Matrix constructor from a Python array
  - u = Matrix([1,2,3])
  - v = Matrix([x**2, y**2, z**2])
- Vectorial functions
  - **Norm**
  - Dot product **dot**() between vectors
  - Cross product **cross**()
- 3D vectors exist in SymPy but will not be used in that lecture (sympy.vector.CoordSys3D).

Vector samples

```
u = sym.Matrix([1,2,3])
v = sym.Matrix([3,2,1])
u.norm()   // norm of a vector
u.dot(v)   // scalar product
u.cross(v) // cross product


ws = sym.symbols("w:4")
w = sym.Matrix(ws)
```

Exercises 4.1 – 4.2: Vectors



Matrices

- Same construction as vectors except that we use an array of array of symbols/values
  - A = sym.Matrix([[1,2],[3,4]])
- **MatrixSymbol** constructor helps to create matrix of symbolic index variables
  - Asymbols = sym.MatrixSymbol ("a", 3, 4)
  - A = sym.Matrix(Asymbols)
  - B = sym.MatrixSymbol("b",4,4).as_explicit()
- **eye**(n) for the identity matrix
- **zeros**() for zero matrix
- **ones**() for one matrix

Matrix operators

- Matrix algebra: * (inner, scalar), +
- Function **det**() gives the determinant
- Function **inv**() gives the inverse matrix, if any
- **eigenvals**() returns the eigenvalues
- **eigenvects**() returns the eigenvectors
- **transpose**() transposes the matrix.
- **diagonalize**() returns the transformation matrix Q and the eigenvalues diagonal matrix L:
  - $A = Q\ L\ Q^{-1}$

Exercises 4.3 – 4.4: Matrices



Linear system of equations

- Problem: Ax = b
- A set of functions helps to solve **linear**() systems of equations in SymPy
- **linsolve**() gives solutions, if any, given a matrix A, a vector b and a list of symbolic variables
  - solutions = sym.linsolve((A,b), (x,y,z))
  - If the system is underdetermined, free variables are returned in solutions
  - A set of equations might be directly given instead of matrix-vector
  - **Linear** systems only!

Miscellaneous functions

- **nsolve**() gives numerical approximation of linear system solutions
  - must provide an initial gues**s**
- **linear_eq_to_matrix**() returns matrix and constant vector of a system of equation

Exercises 4.5: Linear system



Linear differential system

$$x_1'(t) = a_{11}x_1(t) + a_{12}x_2(t)$$
$$x_2'(t) = a_{21}x_1(t) + a_{22}x_2(t)$$
$$X(0)$$
$$X'(t) = AX(t)$$

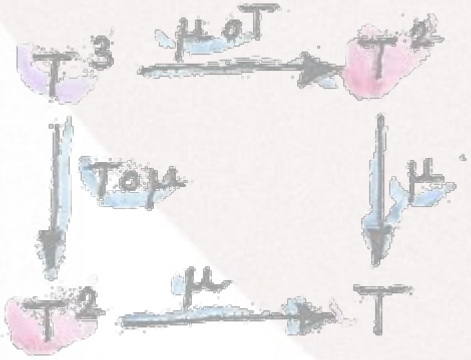
- Algebraic calculation useful to solve temporal linear differential system of equations
- Solutions are given by calculating an exponential of a matrix.

$$e^M = \sum_{n=0}^{\infty} \frac{1}{n} M^n$$
$$X(t) = e^{At}X(0)$$

Exercises 4.5 – 4.6: Linear differential system

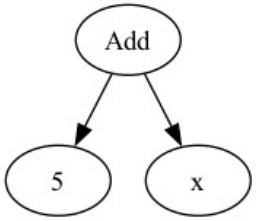


ADVANCED PROCESSING



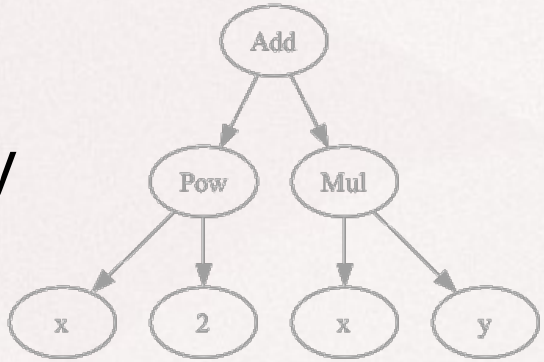
Expression tree

- Expressions in SymPy represented as trees
- Each node of the Abstract Syntactic Tree corresponds to a Python type constructor
- For example, the AST of the expression
  - x + y
  - Add(Symbol("x"), Integer(5))
  - Node Add and Symbol x

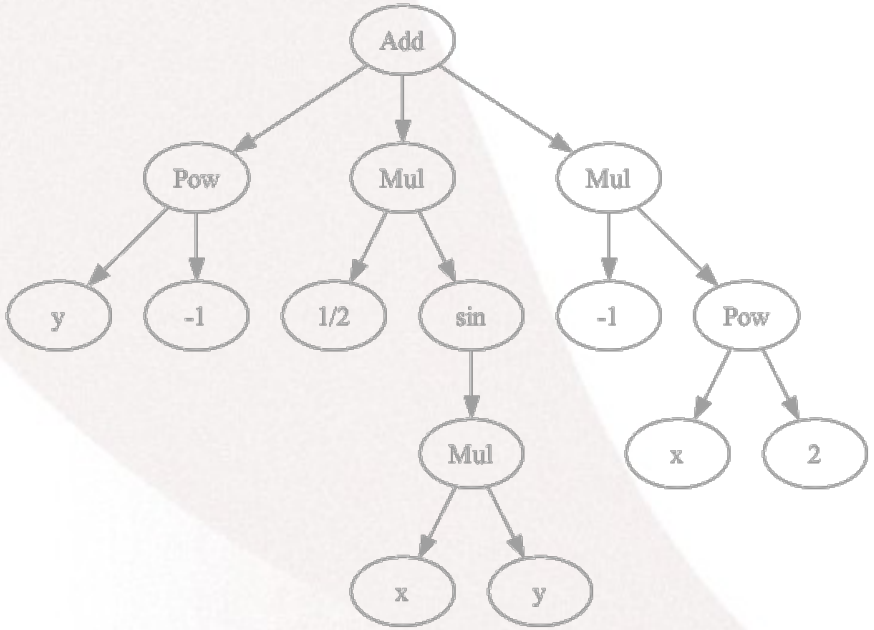


AST examples I

- x**2 + x*y

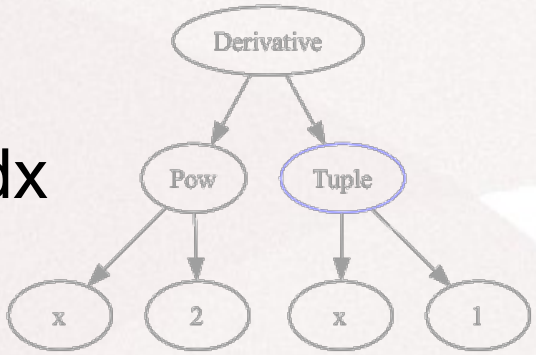


- sin(x*y)/2 - x**2 + 1/y

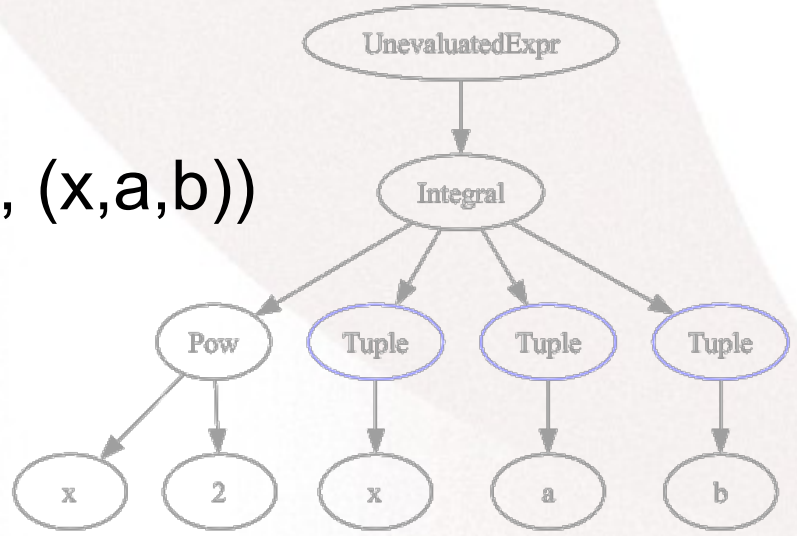


AST examples II

- d(x**2)/dx



- Integral(x**2, (x,a,b))



AST structure

- Every node of the AST has two attributes
  - func:
    - It is the head and type constructor of the object
    - Content of the node
    - (x+y).func = Add()
  - args:
    - Contains the subnodes of the node
    - Arguments of the head constructor
  - Key invariant:
    - expr == expr.func(*expr.args)
- Example:
  - e = x+3
  - e.func = Add
  - e.args = [x,3]

AST creation and evaluation

- When created, by default, AST might be simplified/evaluated
  - $x + x \rightarrow 2x$
- Possible to **prevent** the simplification
  - By parameter:
    - Add(x, y, evaluate: False)
  - By decorator class **UnevaluatedExpr**()
    - expr = UnevaluatedExpr(x+x)
- Possible to **force** evaluation by calling **doit**()
  - expr.doit()

Rendering the AST

- We can see what this expression looks like internally by using **srepr**()
  - sym.srepr(x**2) → Pow(Symbol("x"), Integer(2))
- **latex**() to print the expression as a LaTeX string
  - sym.latex(x+y)
- Representing expression as a graphviz DiGraph with **printdot**()

```
expr = x+y
gDot = sym.dotprint(expr)
g = graphviz.Source(gDot)
g.render('../../output/digraph', format='png')
```

Exercises 5.1 – 5.2: SymPy AST



Traversing the AST

- We can recurse across an expression tree
- The nested nature of args is a perfect fit for recursive functions

```
def printNode(expr):
    print(expr)
    for arg in expr.args:
        printNode(arg)
```

Processing / Rewriting the AST

- During the traversal, the AST can be processed or partly rewritten:
  - Evaluation: 2 + 5 → 7
  - Simplification: x + x → 2x
  - Differentiation
  - …
- Processing / rewriting classes
  - Adapting the processing to the node function when the AST is traversed
  - Overloading methods named according to the node function:
    - process_Add(node)
    - process_Mult(node)
    - ...

Example of processor

```
class AstProcessor:
 def process_node(self, node, parameter = None):
  methodRef = self.process_default
  if isinstance(node,numbers.Number):
   methodRef = self.process_Number
  else:
    methodName = "process_" + node.func.__name__
    methodRef = getattr(self, methodName)
  return methodRef(node, parameter)
 def process_args(self, node, parameter=None)
  return [ pa for a in node.args
    for pa in [self.process_node(a, parameter)]]
 def process_default(self, node, parameter):
  print("process_default called", sym.srepr(node)))
  return node
 def process_Number(self, node, parameter):
  print("process_Number called", node))
  return node
```

Exercises 5.4: Derivation



AST advanced processing

- AST processing and rewriting requires tools:
  - Recognizing patterns in expression
  - Rewriting sub-parts
- Defined in sympy.basic package
- Will be applied to a simple implementation of a user integration class

Atoms

- Atoms are the smallest pieces of an expression: symbols, numbers, number symbols
- **Atoms**() gives the list of atoms in an expression
- Example 1 (complete list)
  - 1+x+2*sin(y+I*pi).Atoms() → {1,2,I,pi,x,y}
- Example 2 (filtered list)
  - 1+x+2*sin(y+I*pi).Atoms(Symbol) → {x,y}

Free symbols

- In most cases, symbols are **free**, except for limits or integrals, where dummy variables are **bound**
- In logic, quantified variables (universal, existential) are bound
- Knowing if a variable is **bound** or **free** is a standard operation in AST rewriting
- **free_symbols**() returns the list of free symbols of an expression

Pattern matching

- A pattern is an expression where may appear meta-variables (Wild). It denotes a sub-language within SymPy language.
- A pattern p matches an expression e if e belongs to the sub-language of p
- **match**() returns a meta-variable valuation matching an expression

```
p_meta = Wild("p_meta")
mq_meta = Wild("p_meta")
expr = (2*x)**y
expr.match((p_meta*x)**q_meta)
```

```
{p_meta:2, q_meta:y}
```

Replace

- **replace**() parses an expression and performs replacement of matching sub-expressions
- Type:
  - (sin(x)+y).replace(sin, cos) → cos(x)+y
- Pattern:
  - a = Wild("a")
  - (sin(2x)+y).replace(sin(a), a) → 2x+y

Rewrite

- **rewrite**() transforms an expression into another, mathematically equivalent, given a rule representing a rewriting **intention**.
- Example 1
  - expr = cos(x) + I*sin(x)
  - expr.rewrite(exp) → exp(I*x)
- Example 2
  - expr.rewrite([cos,sin], exp) → exp(I*x)

Primitive prediction

- Calculating a primitive is a difficult task
- Search by applying combination of heuristics
  - Fundamental primitives
  - Linear property of integration
  - Variable change
  - Integration by parts
  - ...

Exercises 5.5: Primitive



What we have studied...

- Introduction
  - Symbolic calculation
  - SymPy library
- Core instructions
  - Algebraic operators
  - Differential calculation
  - Integral calculation
- Advanced operators
  - Differential equations
  - Laplace transformation
  - Fourier transformation
  - Matrices and vectors
  - AST and expression